FACULTÉ DES SCIENCES ET TECHNOLOGIES
Département Informatique

Université de Lille

MIAGE

MASTER MÉTHODES INFORMATIQUES APPLIQUÉES À LA GESTION DES ENTREPRISES (MIAGE)

Parcours : Ingénierie des Projets Informatiques — Nouvelles Technologies

MÉMOIRE DE PROJET DE FIN D'ÉTUDES

LLM4SQL

Conception et développement d'une plateforme Text-to-SQL

Présenté par :
Mohamed MBENGUE

Sous la direction de :
Mme Anne-Cécile CARON
Mme Julie JACQUES

Année Universitaire :
2025 – 2026

Cadre :
Projet de Fin d'Études (M2)

Remerciements

Je tiens à exprimer ma profonde gratitude à mes encadrantes de projet, Mme Anne-Cécile CARON et Mme Julie JACQUES, pour leur accompagnement bienveillant, leurs orientations scientifiques et techniques précieuses, ainsi que leur grande disponibilité tout au long de ce travail de fin d'études.

Mes remerciements s'adressent également à l'ensemble de l'équipe pédagogique du Master MIAGE de l'Université de Lille (Faculté des Sciences et Technologies) pour la qualité des enseignements dispensés en ingénierie logicielle et en gouvernance des systèmes d'information.

Je remercie enfin chaleureusement mes proches pour leurs encouragements constants, ainsi que mes camarades de promotion pour la richesse de nos échanges et leur soutien tout au long de cette année universitaire.

Déclaration sur l'honneur

Je soussigné, Mohamed MBENGUE, certifie sur l'honneur que ce mémoire de fin d'études constitue un travail personnel de conception et de développement. L'ensemble des bibliothèques logicielles, documentations techniques et publications académiques mobilisées sont rigoureusement référencées.

Résumé

L'accès aux données décisionnelles stockées dans les bases relationnelles d'entreprise reste difficile pour les profils non techniques, qui dépendent des équipes informatiques pour rédiger leurs requêtes SQL. Si les modèles de langage actuels permettent de traduire des questions en langage naturel vers du SQL (*Text-to-SQL*), leur utilisation via des API cloud pose deux problèmes concrets : la fuite potentielle de schémas et de données confidentielles vers des serveurs tiers, et la difficulté de garantir l'exactitude d'un code généré de manière probabiliste.

Ce mémoire présente la conception et la réalisation de **LLM4SQL**, une application d'interrogation de bases de données en langage naturel exécutée intégralement en local sur l'infrastructure de l'entreprise.

Plutôt que d'envoyer directement le schéma complet dans une invite simple (*Zero-shot*), le système s'appuie sur une architecture agentique sous **LangGraph** respectant le protocole ouvert **Model Context Protocol (MCP)**. Conçue pour être agnostique du modèle sous-jacent (Mistral, Llama, Qwen, DeepSeek), la plateforme a été validée avec un modèle local compact (Qwen 3.5:4b servant d'étalon expérimental sobre) qui explore la base étape par étape via quatre outils dédiés : lecture ciblée du schéma, recherche de valeurs textuelles exactes, exécution protégée à blanc (*dry-run*) et boucle de correction en cas d'erreur retournée par PostgreSQL (*Self-Healing*). La sécurité est assurée en amont par l'analyse syntaxique de l'arbre abstrait (AST `sqlglot`), qui bloque toute commande autre qu'un `SELECT`, et par l'usage de sessions transactionnelles en lecture seule.

Évaluée sur un banc d'essai de 18 requêtes représentatives sur une base de référence (`BD Ouvrages`), la démarche d'ingénierie a permis de faire progresser le taux de succès réel (*Execution Accuracy*) de 44,4 % en invite brute à 83,3 % avec l'auto-correction, pour atteindre 100 % grâce à la détection automatique de la nullabilité des clés étrangères et l'injection d'indices de jointures externes (*LEFT JOIN*). L'ensemble fonctionne sur un poste de travail standard sans coût récurrent d'API et dans le respect du confinement des données internes.

Mots-clés : Text-to-SQL, Intelligence Artificielle Locale, LangGraph, Model Context Protocol (MCP), Auto-Correction (Self-Healing), Sécurité des SI, PostgreSQL, Master MIAGE.

Abstract

Accessing business data stored in relational databases remains a bottleneck for non-technical stakeholders who rely on IT teams to write SQL queries. While modern Large Language Models can translate natural language into SQL (*Text-to-SQL*), querying external cloud APIs introduces two critical challenges: the exposure of proprietary database schemas and records to third-party providers, and the difficulty of ensuring the reliability of probabilistically generated code.

This Master's thesis presents the design and implementation of **LLM4SQL**, a natural language database querying platform running entirely locally on on-premise infrastructure.

Rather than injecting entire database schemas into a naive prompt, the system relies on an agentic workflow using **LangGraph** and the open **Model Context Protocol (MCP)** standard. Designed to be model-agnostic (supporting any local LLM such as Llama, Mistral, Qwen, or DeepSeek), the system was benchmarked using a compact local model (Qwen 3.5:4b as an experimental baseline) exploring the database progressively through four specialized tools: targeted schema inspection, categorical text search, non-destructive validation (dry-run), and recursive error recovery driven by PostgreSQL error feedback (*Self-Healing*). Security is strictly verified prior to execution through Abstract Syntax Tree (AST) validation with `sqlglot`, which rejects any non-`SELECT` statement, combined with read-only database transaction scopes.

Evaluated on an 18-query benchmark covering multi-table joins, recursive hierarchies, and aggregations on a reference schema (`BD Ouvrages`), this engineering approach increased the benchmark execution accuracy from 44.4% in zero-shot prompting to 83.3% with self-healing, ultimately reaching 100% through dynamic foreign key nullability inspection and relation-aware join hints (*LEFT JOIN*). The platform runs on standard commodity hardware with zero external API fees while ensuring internal data confinement.

Keywords: Text-to-SQL, Local Artificial Intelligence, LangGraph, Model Context Protocol (MCP), Self-Healing, Information System Security, PostgreSQL, Master MIAGE.

Table des matières

Remerciements 2
Déclaration sur l'honneur 2
Résumé 3
Abstract 4
1. Contexte, Enjeux Stratégiques & État de l'Art 8
1.1. L'Accès aux Données Décisionnelles en Entreprise 8
1.2. Sécurité, Souveraineté et Précision sur le Cadre RGPD 8
1.3. État de l'Art des Approches Text-to-SQL 9
1.4. Analyse Économique et Méthodologie du Coût Global (TCO) 10
1.4.1. 1. Grille Tarifaire des Modèles (Relevé Code Arena / WebDev, Août 2026) 10
1.4.2. 2. Hypothèses de Charge d'une PME 10
1.4.3. 3. Modélisation Comparative sur 3 Ans 11
2. Organisation du Projet, Gouvernance & Environnement Technique 11
2.1. Cadre Organisationnel et Rôles (MOA / MOE) 11
2.2. Démarche Méthodologique et Discipline Qualité (TDD) 12
2.3. Matrice de Gestion des Risques Projet & Plans de Continuité 12
2.4. Environnement Technique et Outillage Retenu 13
2.5. Découpage et Chronologie des Sprints 14
3. Spécifications Fonctionnelles & Modélisation des Données 15
3.1. Analyse des Besoins et Personas Utilisateurs 15
3.2. Spécifications Fonctionnelles (Matrice MoSCoW) 15
3.3. Exigences Non-Fonctionnelles & Métriques Cibles 16
3.4. Modèle Relationnel de Référence : BD Ouvrages 17
4. Conception Architecturale & Modélisation du Système 18
4.1. Topologie Globale et Découpage en Couches 18
4.2. Le Moteur Agentique : Cycle de Décision ReAct et Auto-Correction (Self-Healing) 19
4.3. Le Standard MCP comme Couche de Gouvernance 20
4.4. Contrôles de Sécurité à Trois Niveaux 20
4.4.1. Niveau 1 : Analyse Syntaxique par Arbre Abstrait (AST) 21
4.4.2. Niveau 2 : Isolation Transactionnelle au Niveau du Pilote PostgreSQL ... 21
4.4.3. Niveau 3 : Partitionnement Mémoire du Cache Sémantique (OWASP LLM08) 21
4.5. Architecture du Frontend et Optimisations de Rendu 22
5. Implémentation Technique & Défis d'Ingénierie Résolus 22

5.1. Communication Temps Réel et Explicabilité Cognitive (XAI) 22
5.2. Cache Sémantique Vectoriel en Mémoire & Bypass Dynamique 23
5.3. Décisions d'Architecture Critiques : Impasses Abandonnées vs Solutions Adoptées 23
5.3.1. 1. L'Impasse de la Traduction AST ➔ Français (Phase R16) 24
5.3.2. 2. L'Impasse du Dynamic Few-Shot RAG par Gabarits Abstraits (Phase R27) 25
5.3.3. 3. L'Impasse de la Génération de Diagrammes Mermaid par LLM (Phase 5) 25
5.4. Défis d'Ingénierie Logicielle et Solutions Implémentées 25
5.4.1. Défi 1 : L'Effondrement d'Attention (Attention Collapse) et la Directivité du Prompt 26
5.4.2. Défi 2 : Faux Négatifs d'Évaluation & Algorithme de Soft-Matching par Jetons 26
5.4.3. Défi 3 : La Détection Universelle de Nullabilité et les Join Hints dans MCP 27
5.4.4. Défi 4 : Migration Asynchrone vers psycopg v3 et Pool Thread-Safe 28
6. Évaluation Expérimentale, Benchmarks & Résultats 28
6.1. Protocole Expérimental et Jeu d'Épreuve Canonique 28
6.2. Moteur d'Execution Accuracy et Normalisation Sémantique 29
6.3. Résultats Quantitatifs : Étude d'Ablation à Modèle Constant 29
6.4. Autopsie d'un Échec Critique : Le Piège de la Jointure Interne (Cas 6) 30
6.4.1. La Problématique du Cas 6 30
6.4.2. Pourquoi le Modèle et le Self-Healing Échouaient-ils ? 30
6.4.3. La Solution d'Ingénierie : L'Injection de Join Hints via MCP 31
6.5. Exemple Concret d'Exécution de Bout en Bout 31
7. Bilan Personnel, Perspectives & Conclusion 32
7.1. Retour d'Expérience et Difficultés Concrètes de Développement 32
7.1.1. L'Impasse de l'Évaluation Sémantique par AST (Le Grand Pivot) 32
7.1.2. L'Enquête sur le Cas 6 : Le Piège de la Clé Étrangère Optionnelle 33
7.1.3. L'Échec du Dynamic Few-Shot par Gabarits Synthétiques 33
7.1.4. La Règle d'Or de l'Ingénierie : Ne pas solliciter l'IA pour ce que le code fait mieux 33
7.1.5. L'Apprivoisement des Petits Modèles Locaux (4B) 34
7.2. Bilan des Compétences Développées 34
7.3. Perspectives d'Évolution 34
7.4. Projet Professionnel 35
7.5. Conclusion Générale 35

Références Bibliographiques 36
Annexes 37
7.6. Annexe A : Schéma DDL de la Base de Données de Référence (BD Ouvrages) . 37
7.7. Annexe B : Jeu d'Épreuve Canonique de Benchmark (18 Scénarios) 39
7.8. Annexe C : Spécification des Outils du Serveur MCP 41

1. Contexte, Enjeux Stratégiques & État de l'Art

1.1. L'Accès aux Données Décisionnelles en Entreprise

Dans les systèmes d'information décisionnels, la base de données relationnelle constitue le référentiel des données opérationnelles indispensables au pilotage de l'activité. Toutefois, l'exploitation quotidienne de ces données crée une friction organisationnelle classique entre deux profils aux compétences complémentaires :

D'une part, les **décideurs et responsables métiers** possèdent la compréhension des indicateurs de gestion, mais ne maîtrisent pas la syntaxe technique du langage SQL. Chaque besoin ponctuel d'information implique la création d'un ticket auprès de la direction des systèmes d'information (DSI), ce qui introduit des délais de traitement de plusieurs heures à plusieurs jours et freine l'exploration spontanée des données.

D'autre part, les **équipes data et développeurs** connaissent les schémas relationnels, mais consacrent une part significative de leur temps à rédiger manuellement des requêtes SQL répétitives (jointures multiples, agrégations, filtres de dates). Pour ces profils techniques, formuler une intention en français pour obtenir un premier jet de requête syntaxiquement valide représente un gain de temps appréciable, à condition de pouvoir inspecter et modifier directement le code produit avant son exécution (*Human-in-the-Loop*).

Cette double attente justifie la mise en place d'un système de traduction du langage naturel vers SQL (**Text-to-SQL**), conçu à la fois comme une interface d'interrogation autonome pour les métiers et comme un assistant de productivité pour les profils techniques.

1.2. Sécurité, Souveraineté et Précision sur le Cadre RGPD

Le recours aux modèles de fondation hébergés en mode SaaS via des API distantes (tels que ceux d'OpenAI, d'Anthropic ou de Google) soulève des questions de sécurité et de conformité qui méritent une analyse juridique et technique rigoureuse :

1. **Exposition des structures et des données de gestion** : Pour générer une requête SQL exacte, le modèle de langage doit recevoir la définition DDL des tables (colonnes, types, clés primaires et étrangères) ainsi que des échantillons de valeurs textuelles pour résoudre les filtres métiers. L'envoi de ces métadonnées et données à des serveurs tiers expose des informations potentiellement sensibles ou protégées par le secret d'affaires.
2. **Transferts de données hors UE et Chapitre V du RGPD** : L'utilisation d'API hébergées sur des infrastructures soumises à des juridictions extraterritoriales (notamment le *Cloud Act* américain) pose la question de la conformité aux articles 44 et suivants du RGPD relatifs aux transferts internationaux de données. L'exécution 100 % locale sur l'infrastructure de l'entreprise résout ce point spécifique en garantissant que les don-

nées ne franchissent jamais le périmètre du réseau interne et dispense l'organisation de souscrire des contrats de sous-traitance de traitement au sens de l'article 28.

3. **Obligations organisationnelles résiduelles** : L'exécution locale ne dispense toutefois pas l'entreprise d'appliquer les autres obligations du RGPD : le contrôle des droits d'accès à la base de données (RBAC), le chiffrement des disques au repos, la journalisation des requêtes exécutées pour la traçabilité des accès, et la gestion des droits des personnes concernées (articles 15 à 22).

Le choix d'une architecture locale (**air-gapped**) permet ainsi d'éliminer le risque de fuite vers des tiers tout en s'intégrant dans la politique de sécurité générale du système d'information de l'organisation.

1.3. État de l'Art des Approches Text-to-SQL

La littérature en traitement du langage naturel et en bases de données met en évidence trois grandes approches pour le *Text-to-SQL* :

Approche	Principe technique	Limites identifiées	Statut
Invite directe (*Zero-shot*)	Insertion brute du DDL complet et de la question dans le prompt système.	Hallucinations fréquentes sur les noms de colonnes et saturation du contexte sur les bases complexes.	Inadapté en production
Spécialisation (*Fine-tuning*)	Réentraînement supervisé d'un modèle sur des paires question-requête SQL.	Coût de calcul élevé et nécessité de réentraîner le modèle à chaque modification du schéma de données.	Coûteux à maintenir
Architecture agentique (`ReAct + MCP`)	Orchestration par graphe d'états [1] avec outils normalisés [2] pour inspecter, tester (*dry-run*) et corriger ([3]).	Nécessite la mise en place de protocoles stricts et de règles de validation robustes.	Standard retenu (LLM4SQL)

Tableau 1. – Comparatif des approches de génération Text-to-SQL dans la littérature scientifique

Les benchmarks de recherche comme **Spider** [4] et **BIRD-SQL** [5] indiquent qu'une invite textuelle simple ne suffit pas pour traiter des bases relationnelles réelles. Les architectures modulaires décomposées (**DIN-SQL** [6]) s'imposent aujourd'hui en séparant l'exploration du schéma de la phase de génération et de validation.

1.4. Analyse Économique et Méthodologie du Coût Global (TCO)

Pour évaluer la pertinence économique d'une solution locale par rapport aux offres d'API distantes, nous avons modélisé le Coût Total de Possession (**Total Cost of Ownership - TCO**) sur un horizon de 3 ans.

1.4.1. 1. Grille Tarifaire des Modèles (Relevé Code Arena / WebDev, Août 2026)

Sur la base du classement officiel **Code Arena | WebDev** [7] (relevé du 22 août 2026 portant sur 118 modèles évalués sur des flux de code agentiques avec raisonnement multi-étapes), les coûts d'API par million (M) de tokens se répartissent selon trois gammes principales :

- **Modèles de pointe (Frontier Tier)** :
  - *Claude Opus 5 Max* (Anthropic, Score 1691) : 5,00 $ / M tokens d'entrée, 25,00 $ / M tokens de sortie.
  - *GPT-5.6 Sol xHigh* (OpenAI, Score 1619) : 4,00 $ / M tokens d'entrée, 20,00 $ / M tokens de sortie.
- **Modèles intermédiaires réactifs (Mid-Range Tier)** :
  - *Qwen 3.8 Max* (Alibaba, Score 1669) : 2,00 $ / M tokens d'entrée, 6,00 $ / M tokens de sortie.
  - *Gemini 3.7 Flash High* (Google, Score 1587) : 0,75 $ / M tokens d'entrée, 3,57 $ / M tokens de sortie.
- **Modèles économiques (Budget Tier)** :
  - *DeepSeek V4 Flash High* (DeepSeek, Score 1579) : 0,44 $ / M tokens d'entrée, 1,32 $ / M tokens de sortie.
  - *Qwen 3.8 27B* (Alibaba, Score 1595) : 0,50 $ / M tokens d'entrée, 3,00 $ / M tokens de sortie.

1.4.2. 2. Hypothèses de Charge d'une PME

- **Volume d'activité** : Une équipe de 10 collaborateurs posant chacun 15 requêtes analytiques par jour ouvré, sur 220 jours ouvrés par an, soit un volume de **33 000 requêtes décisionnelles par an**.
- **Consommation par cycle agentique** : Chaque requête en langage naturel nécessite en moyenne 2 à 3 tours d'interaction (introspection du schéma MCP, test de validation à blanc *dry-run*, et génération finale). Le contexte moyen cumulé est estimé à **6 000 tokens d'entrée** (DDL des tables, historique, consignes) et **800 tokens de sortie** (réflexion et code SQL).
- **Volume annuel de tokens** : $33,000 \times 6,000 = 198$ millions de tokens d'entrée et $33,000 \times 800 = 26\{,\}4$ millions de tokens de sortie par an.

1.4.3. 3. Modélisation Comparative sur 3 Ans

Poste de charge	**Frontier API (GPT-5.6 / Opus 5)**	**Mid-Range API (Gemini 3.7 Flash)**	**Budget API (DeepSeek V4 Flash)**	**Solution Locale LLM4SQL (Modèle local, ex. Qwen 4B)**
Tarif / 1M tokens	4 à 5 $ In / 20 à 25 $ Out	0,75 $ In / 3,57 $ Out	0,44 $ In / 1,32 $ Out	**0,00 € (Inférence locale)**
Coût direct annuel tokens	1 320 à 1 650 $ (1 220 à 1 530 €)	243 $ (225 €)	122 $ (113 €)	**0,00 €**
Coût marginal électricité	Inclus dans le service	Inclus dans le service	Inclus dans le service	15 à 25 € / an (sur matériel existant)
Confidentialité du schéma	Données transmises à un tiers	Données transmises à un tiers	Données transmises à un tiers	**100 % interne au réseau**
Coût d'exploitation 3 ans	**3 660 à 4 590 €**	**675 €**	**340 €**	**45 à 75 € (Électricité)**

Tableau 2. – Modélisation chiffrée du TCO sur 3 ans (33 000 requêtes/an selon les tarifs officiels Code Arena Août 2026)

Cette modélisation démontre que même face aux API économiques de 2026 (comme DeepSeek V4 Flash), la solution locale `LLM4SQL` reste la seule option garantissant un coût d'exploitation marginal quasi nul tout en empêchant toute fuite des schémas de base de données d'entreprise et des filtres de requêtes vers des serveurs tiers.

2. Organisation du Projet, Gouvernance & Environnement Technique

2.1. Cadre Organisationnel et Rôles (MOA / MOE)

Bien que mené dans le cadre académique d'un projet de fin d'études de Master MIAGE, le projet LLM4SQL a été structuré selon une gouvernance professionnelle formalisant la séparation stricte entre la maîtrise d'ouvrage (MOA) et la maîtrise d'œuvre (MOE) :

- **Maîtrise d'ouvrage (Cadrage stratégique et validation)** : Les encadrantes universitaires de projet (Mme Anne-Cécile CARON et Mme Julie JACQUES) ont fixé les exigences scientifiques et fonctionnelles, défini les critères d'acceptabilité de l'évaluation et fourni le schéma relationnel de référence (`BD Ouvrages`) représentatif des difficultés d'interrogation en entreprise.

- **Maîtrise d'œuvre (Conception, réalisation et tests)** : Nous avons assuré la responsabilité intégrale de la chaîne d'ingénierie logicielle : étude de l'état de l'art, arbitrages d'architecture, implémentation full-stack (FastAPI, React 19, LangGraph, serveur MCP), sécurisation applicative et industrialisation des bancs de test automatisés.

Des jalons réguliers de revue de sprint ont permis de valider les incréments logiciels, tandis que les résultats empiriques des premières expérimentations ont guidé les choix d'architecture (notamment l'abandon de l'évaluation syntaxique au profit d'un moteur d'*Execution Accuracy* sur base réelle).

2.2. Démarche Méthodologique et Discipline Qualité (TDD)

Afin d'éviter les régressions et d'assurer une robustesse logicielle continue, le projet a appliqué une méthodologie itérative et incrémentale couplée à une discipline rigoureuse de développement piloté par les tests (*Test-Driven Development*) :

1. **Établissement d'une référence (*Baseline*)** : Avant tout refactoring ou ajout de fonctionnalité, la suite complète de vérification (tests unitaires backend sous `pytest` et contrôle de compilation TypeScript strict) est exécutée pour figer l'état nominal.
2. **Développement chirurgical et modulaire** : Les modifications sont appliquées selon le principe de responsabilité unique (*Single Responsibility Principle* - SRP), isolant les couches API, moteur agentique, serveur d'outils et composants graphiques.
3. **Vérification de non-régression** : La suite de tests est immédiatement réexécutée. Un incrément n'est accepté que si 100% des tests précédemment validés restent au vert et que les nouveaux cas de test d'acceptation sont couverts.
4. **Validation statique continue** : Le code frontend est soumis à une configuration TypeScript en mode strict (`strict: true`) combinée à ESLint pour bannir les types permissifs (`any`) et prévenir les fuites de mémoire.

2.3. Matrice de Gestion des Risques Projet & Plans de Continuité

Conformément aux pratiques d'alignement stratégique des systèmes d'information, nous avons formalisé une matrice des risques techniques et organisationnels, assortie de mesures de mitigation concrètes :

Risque identifié	Probabilité	Impact	Mesure de mitigation appliquée dans LLM4SQL
Hallucinations de tables/colonnes par le LLM	Élevée	Critique	Introspection dynamique via `mcp_get_schema` (*schema pruning*) et injection stricte des métadonnées DDL réelles dans le contexte.
Exécution de commandes destructives (`DROP`, `DELETE`)	Moyenne	Critique	Défense en profondeur à 2 barrières : rejet syntaxique par AST `sqlglot` (SELECT-only) et sessions verrouillées en `readonly=True` sous `psycopg` v3.
Saturation mémoire / effondrement d'attention du 4B	Élevée	Majeur	Prompt système condensé en anglais technique, suppression du few-shot verbeux et transmission d'échantillons bornés (`limit=3`).
Indisponibilité ou plantage du serveur LLM local	Faible	Majeur	Détection temps réel de santé de l'API Ollama avec indicateur de statut visuel dans l'interface et gestion d'exceptions réseau non bloquantes.
Pollution de données entre schémas Sandbox et Prod	Moyenne	Majeur	Partitionnement strict du cache sémantique vectoriel par `target_schema` conformément au standard OWASP LLM08 [8].

Tableau 3. – Matrice des risques projet et mesures de mitigation du système d'information

2.4. Environnement Technique et Outillage Retenu

Le tableau ci-dessous synthétise la pile technologique mise en œuvre pour assurer des performances maximales et une reproductibilité totale du déploiement :

Composant	Technologie	Rôle et justification technique
Gestionnaire Python	`uv`	Gestionnaire de paquets ultra-rapide en Rust, garantissant la reproductibilité des environnements via un fichier de verrouillage strict (`uv.lock`).
Backend & API	`FastAPI` / `Uvicorn`	Serveur asynchrone Python haute performance avec validation automatique des schémas via `Pydantic v2` et streaming d'événements `SSE`.
Moteur Agentique	`LangGraph`	Framework de graphes d'états récursifs modélisant le cycle de raisonnement ReAct [1] et les conditions de branchement d'auto-correction.
Serveur d'Outils	Standard `MCP`	Protocole ouvert de gouvernance [2] isolant les accès à la base de données à travers 4 outils strictement typés.
Moteur LLM Local	`Ollama`	Serveur d'inférence open-source exécutant le modèle quantifié avec accélération GPU et calcul d'embeddings vectoriels.
Base de Données	`PostgreSQL 16`	SGBD relationnel robuste conteneurisé sous `Docker` avec pilote `psycopg` v3 en mode pool de connexions asynchrone.
Frontend SPA	`React 19` / `Vite`	Interface utilisateur moderne en `TypeScript` strict avec virtualisation de données `@tanstack/react-virtual` et styles `Tailwind CSS v4`.
Assurance Qualité	`pytest`, `ESLint`	Suite de 85 tests automatisés backend et validation statique rigoureuse du code source frontend.

Tableau 4. – Environnement technique et outillage de la plateforme LLM4SQL

2.5. Découpage et Chronologie des Sprints

Le développement du projet s'est déroulé en cinq phases méthodologiques :

1. **Phase 1 : Cadrage, BDD et Conteneurisation** : Initialisation du conteneur PostgreSQL Docker, création du script de peuplement `seed_db.py` et validation des premières connexions avec Ollama.
2. **Phase 2 : Serveur MCP et Moteur Agentique** : Implémentation du serveur d'outils sécurisé, développement du graphe d'états LangGraph et boucle de Self-Healing.

3. **Phase 3 : Interface Utilisateur et Restitution** : Conception de la SPA React 19, bac à sable DDL (Sandbox), parser de diagrammes Mermaid et explorateur tabulaire avec export CSV.
4. **Phase 4 : Sécurité AST et Banc d'Évaluation** : Intégration du parseur syntaxique SQLGlot [9], développement du service d'Execution Accuracy et du soft-matching.
5. **Phase 5 : Optimisations, Join Hints et Industrialisation** : Résolution du Lossy Join (indices de nullabilité MCP), virtualisation des tableaux volumineux et conteneurisation Docker Compose unifiée.

3. Spécifications Fonctionnelles & Modélisation des Données

3.1. Analyse des Besoins et Personas Utilisateurs

Afin de répondre aux exigences hétérogènes de l'entreprise, trois profils utilisateurs types ont guidé la conception fonctionnelle de la plateforme :

- **Le Décideur et Responsable Métier (Persona 1)** : Ne disposant pas de compétences techniques en `SQL`, il souhaite interroger la base de données directement en français (« Quels sont les ouvrages parus après 2010 ? ») et obtenir instantanément un tableau de bord lisible, complété par des visualisations graphiques et des fonctions d'export (CSV).
- **L'Analyste de Données et Développeur (Persona 2)** : Maîtrisant parfaitement la syntaxe relationnelle, il recherche un accélérateur de productivité pour générer automatiquement les requêtes complexes comportant de multiples jointures. Il exige une transparence totale sur le code généré, la capacité de l'éditer directement dans l'interface et de le ré-exécuter sans passer par l'IA (*Human-in-the-Loop*).
- **L'Administrateur de Base de Données / Responsable Sécurité (Persona 3)** : Garant de l'intégrité du système d'information, il impose des contraintes non négociables : étanchéité totale face aux fuites de données vers l'extérieur, interdiction absolue de toute modification de structure ou de contenu (`SELECT`-only) et cloisonnement étanche des espaces de test.

3.2. Spécifications Fonctionnelles (Matrice MoSCoW)

Le périmètre applicatif a été hiérarchisé selon la méthodologie MoSCoW afin de cadencer les livraisons :

Priorité	Exigence Fonctionnelle	Composant Associé
Must Have	Traduction fiable de questions en français vers des requêtes `PostgreSQL`.	Graphe `LangGraph` & `Ollama`
Must Have	Boucle d'auto-correction récursive en cas d'erreur de syntaxe ou de schéma (*Self-Healing*).	Outils `MCP` & Graphe d'états
Must Have	Garantie d'exécution strictement en lecture seule (rejet des écritures).	AST `sqlglot` & `psycopg` v3
Must Have	Restitution tabulaire interactive avec tri dynamique, pagination et export CSV.	Composant `DataTable`
Must Have	Bac à sable DDL (*Sandbox*) permettant de tester des schémas personnalisés en isolation.	Panneau `SandboxPanel`
Should Have	Modélisation relationnelle instantanée du schéma DDL en diagramme entité-association.	Parser AST Python pur
Should Have	Banc de test automatisé avec calcul d'exactitude d'exécution (*Execution Accuracy*).	Module `BenchmarkPanel`
Should Have	Diffusion en direct des étapes de réflexion de l'agent (`reasoning_content` / `<think>`).	Flux `SSE` & Accordéon XAI
Could Have	Édition directe du code SQL généré dans le chat avec ré-exécution immédiate.	Composant `SqlCodeBlock`
Could Have	Cache sémantique vectoriel en mémoire pour accélérer les requêtes récurrentes (0 ms).	Module de cache d'embeddings

Tableau 5. – Matrice des spécifications fonctionnelles selon la méthode MoSCoW

3.3. Exigences Non-Fonctionnelles & Métriques Cibles

Les contraintes de qualité logicielle, de sécurité et d'ergonomie couvrent quatre axes majeurs :

- **Souveraineté et Confidentialité** : L'intégralité du traitement (inférence du LLM, analyse syntaxique, requêtage BDD, rendu UI) s'exécute strictement en local (**air-gapped**), sans aucun flux réseau sortant vers l'extérieur.
- **Performance et Latence** : Le temps de réponse moyen doit rester compris entre **3 et 5 secondes** pour les requêtes complexes multi-jointures, et tomber sous les **100 ms** en cas de correspondance dans le cache sémantique vectoriel.

- **Agnosticisme et Sobriété Matérielle** : L'architecture est totalement agnostique du modèle sous-jacent et supporte n'importe quel LLM local via Ollama (Llama, Mistral, Qwen, DeepSeek). Dans cette étude, un modèle compact quantifié (`qwen3.5:4b`) a servi de référence minimale sous contrainte de ressources (< 8 Go de VRAM) pour prouver la viabilité sur du matériel standard, sachant que des modèles plus volumineux améliorent proportionnellement la vitesse et l'aisance de raisonnement.
- **Sécurité Applicative** : Rejet systématique de toute commande autre qu'un `SELECT` via l'analyse de l'arbre syntaxique (`AST`) de `sqlglot` [9], indépendamment du texte généré par le LLM.

3.4. Modèle Relationnel de Référence : BD Ouvrages

Pour concevoir et valider la plateforme sur des requêtes représentatives de la complexité des bases d'entreprise, la maîtrise d'ouvrage a mis à disposition le schéma relationnel de référence `BD Ouvrages`. Ce schéma modélise une bibliothèque d'œuvres éditoriales :

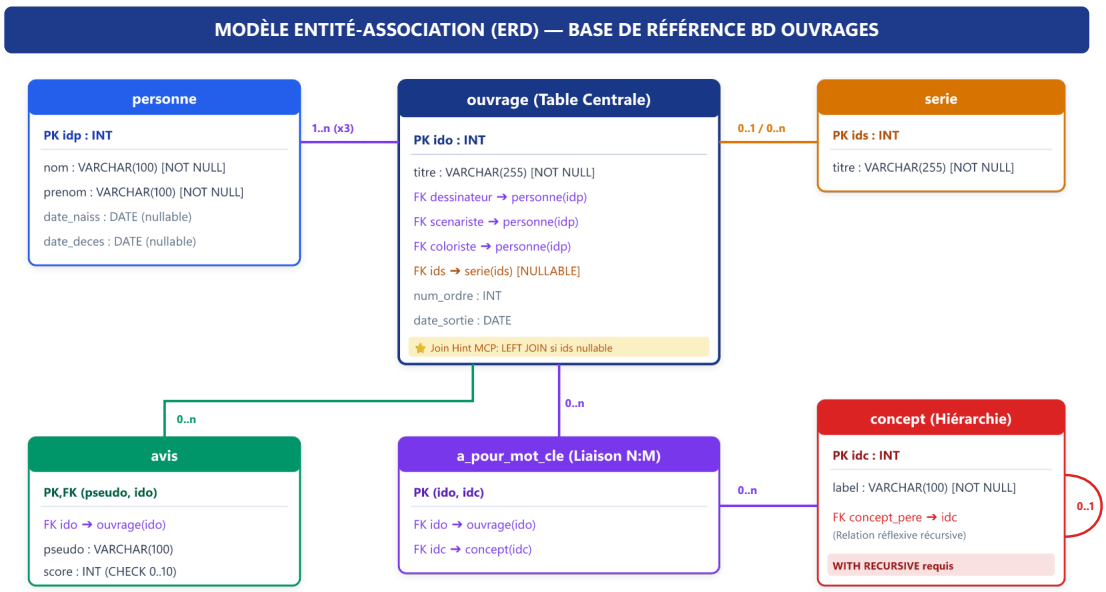


Fig. 1. – Modèle Entité-Association (ERD) de la base de données de référence BD Ouvrages

Ce schéma a été spécifiquement conçu pour mettre à l'épreuve les capacités de raisonnement du système face à trois pièges relationnels classiques :

1. **Clés étrangères multiples vers une même table cible** : La table `ouvrage` référence trois fois la table `personne` (`dessinateur`, `scenariste`, `coloriste`), ce qui impose à l'agent de réaliser des auto-jointures multiples avec des alias de tables explicites (`JOIN personne pd ON o.dessinateur = pd.idp JOIN personne ps ON o.scenariste = ps.idp`).
2. **Clé étrangère optionnelle (Nullable FK)** : Le champ `ids` reliant un album à sa série est optionnel (valeur `NULL` pour les ouvrages hors-série dits *one-shots*). Une jointure

interne naïve (`INNER JOIN`) élimine silencieusement les ouvrages sans série, exigeant impérativement l'application d'un `LEFT JOIN`.

3. **Hiérarchie réflexive récursive** : La table `concept` possède une clé réflexive `concept_pere` pointant vers son propre identifiant `idc`, nécessitant des requêtes récursives complexes (`WITH RECURSIVE`) pour explorer les sous-arborescences thématiques.

4. Conception Architecturale & Modélisation du Système

4.1. Topologie Globale et Découpage en Couches

L'architecture logicielle de `LLM4SQL` est structurée en quatre couches bien séparées afin de découpler l'interface utilisateur, l'API d'exposition, l'orchestration de l'agent et la persistance des données :

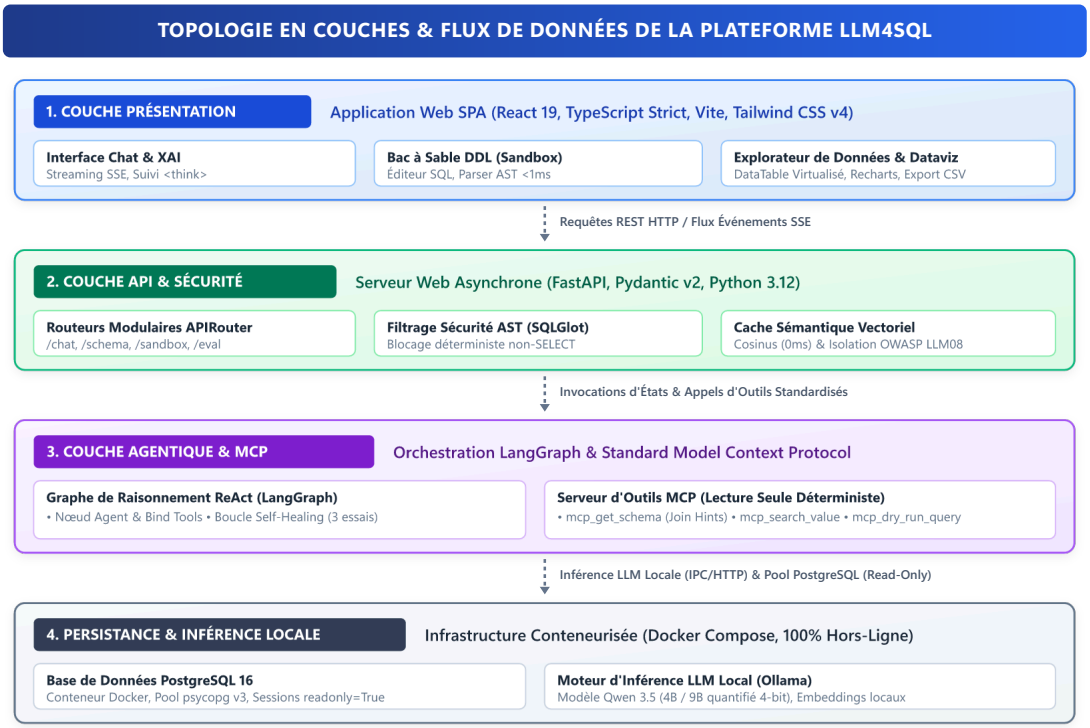


Fig. 2. – Topologie globale en couches et flux de données de la plateforme LLM4SQL

Chaque couche assure un rôle précis au sein de l'application :

1. **Couche Présentation (SPA React 19 & TypeScript)** : Interface web développée avec `Vite` et `Tailwind CSS v4`. Elle intègre la réception du flux d'événements serveur (`SSE`), la modélisation relationnelle de schémas, le bac à sable DDL (*Sandbox*) et la virtualisation des jeux de données volumineux via `@tanstack/react-virtual`.

2. **Couche Exposition API & Sécurité (FastAPI)** : Serveur web asynchrone en Python 3.12. Il assure la validation stricte des données entrantes/sortantes via `Pydantic v2`, le filtrage syntaxique par arbre abstrait (`AST`), le rate-limiting avec `slowapi` et le partitionnement du cache sémantique vectoriel.
3. **Couche Agentique & Gouvernance (LangGraph & Serveur MCP)** : Moteur de décision structurant la boucle ReAct [1] et l'auto-correction en cas d'erreur. Les accès à la base de données sont gérés par un serveur d'outils conforme au standard ouvert **Model Context Protocol (MCP)** [2].
4. **Couche Persistance & Inférence Locale (PostgreSQL 16 & Ollama)** : Infrastructure locale conteneurisée sous `Docker Compose`. Le serveur `Ollama` exécute le modèle de langage local sélectionné (Mistral, Llama, DeepSeek, Qwen), tandis que la base de données `PostgreSQL 16` est interrogée via un pool asynchrone `psycopg` v3 avec des transactions en lecture seule (`readonly=True`).

4.2. Le Moteur Agentique : Cycle de Décision ReAct et Auto-Correction (Self-Healing)

Plutôt que d'envoyer le schéma complet dans une invite unique (*Zero-shot*), ce qui surcharge la mémoire du modèle et provoque des hallucinations, `LLM4SQL` découpe le raisonnement sous la forme d'un graphe d'états récursif sous `LangGraph` :

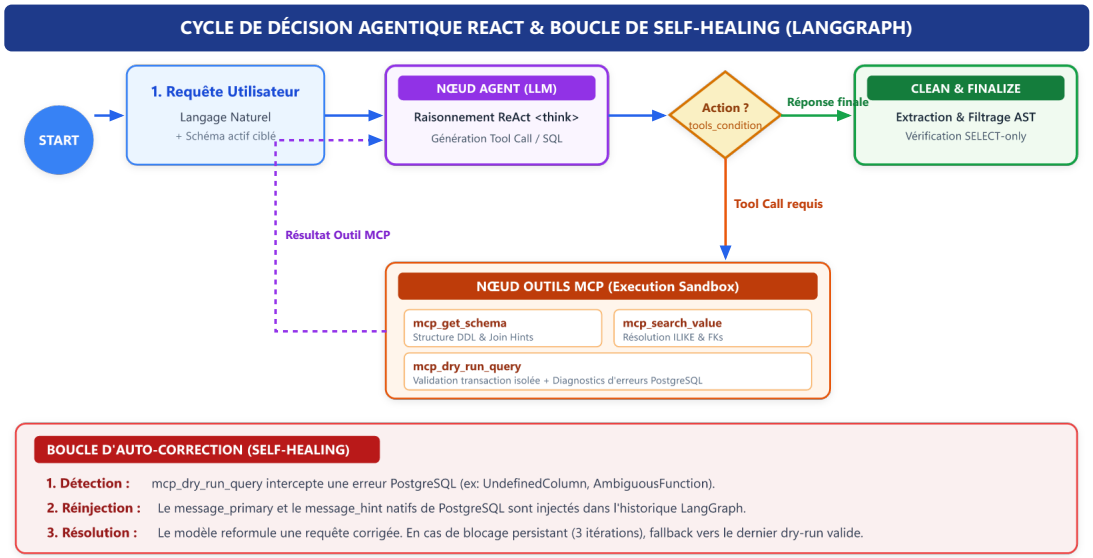


Fig. 3. – Cycle de décision cognitive ReAct et boucle d'auto-correction (Self-Healing) sous LangGraph

Le graphe orchestre quatre étapes successives :

- **Étape 1 : Introspection ciblée du schéma (*Schema Pruning*)** : L'agent appelle `mcp_get_schema` pour récupérer uniquement les tables nécessaires à la question. Cette sélection préalable évite de saturer la fenêtre de contexte du modèle.
- **Étape 2 : Résolution des entités et valeurs textuelles** : Pour les requêtes comportant des critères textuels sur des noms propres ou des concepts (« ouvrages dessinés par Uderzo »), l'outil `mcp_search_value` effectue une recherche `ILIKE` en base et transmet au modèle la casse exacte et les clés étrangères associées.
- **Étape 3 : Validation protégée par exécution à blanc (*Dry-Run*)** : Avant de renvoyer le résultat à l'utilisateur, la requête SQL est testée dans une transaction isolée via `mcp_dry_run_query` pour vérifier l'exactitude syntaxique et la cohérence des types.
- **Étape 4 : Boucle d'Auto-Correction (*Self-Healing*)** : Si PostgreSQL signale une erreur (ex. `UndefinedColumn`, alias ambigu), le message d'erreur et les indications fournies par la base (`message_hint`) sont réinjectés dans le graphe. Le modèle génère alors une version corrigée (jusqu'à 3 itérations). En cas d'échec persistant, un mécanisme de repli bascule automatiquement sur le dernier état valide du dry-run.

4.3. Le Standard MCP comme Couche de Gouvernance

L'adoption du standard **Model Context Protocol (MCP)** [2] permet de séparer proprement le modèle de langage de la couche d'accès à la base de données. Le serveur MCP expose quatre outils spécialisés :

1. `mcp_get_schema(table_name, target_schema)` : Renvoie la structure des colonnes, types PostgreSQL, contraintes PK/FK, statut de nullabilité et indices de jointures relationnelles (*Join Hints*).
2. `mcp_search_value(query, target_table, target_schema)` : Recherche les correspondances textuelles via `ILIKE` et fournit des suggestions automatiques de jointures sur clés étrangères (`foreign_key_suggestion`).
3. `mcp_get_sample_data(table_name, limit=3)` : Fournit un extrait restreint de 3 enregistrements pour permettre au modèle d'appréhender le formatage réel des données sans surcharger le contexte.
4. `mcp_dry_run_query(sql, target_schema)` : Exécute une validation à blanc en transaction fermée et renvoie le statut de réussite, la liste des colonnes résultantes et les diagnostics PostgreSQL (`message_primary`, `message_detail`, `message_hint`).

4.4. Contrôles de Sécurité à Trois Niveaux

Pour préserver l'intégrité de la base de données et empêcher l'exécution de requêtes destructrices (volontaires ou accidentelles), `LLM4SQL` applique un contrôle à trois niveaux complémentaires :

4.4.1. Niveau 1 : Analyse Syntaxique par Arbre Abstrait (AST)

Toute requête produite par l'agent ou saisie manuellement par l'utilisateur est analysée par la bibliothèque `sqlglot` [9] avant d'atteindre la base de données. Si l'arbre syntaxique contient une commande de modification (`INSERT`, `UPDATE`, `DELETE`, `DROP`, `ALTER`, `TRUNCATE`, `GRANT`), la requête est immédiatement rejetée :

```
# Extrait du validateur de sécurité (app/services/agent/security.py)
import sqlglot
from sqlglot import exp

FORBIDDEN_EXPRESSIONS = (
    exp.Insert, exp.Update, exp.Delete, exp.Drop,
    exp.Alter, exp.Create, exp.Command, exp.TruncateTable
)

def validate_sql_query(sql: str) -> None:
    """Analyse l'arbre syntaxique (AST) et bloque toute commande non-SELECT."""
    try:
        parsed_statements = sqlglot.parse(sql, read="postgres")
    except Exception as e:
        raise ValueError(f"Syntaxe SQL invalide : {str(e)}")

    for statement in parsed_statements:
        if statement is None:
            continue
        # Vérifie récursivement l'absence d'expressions mutatrices dans l'AST
        for forbidden in FORBIDDEN_EXPRESSIONS:
            if statement.find(forbidden):
                raise PermissionError("Sécurité : Seules les requêtes SELECT en
lecture seule sont autorisées.")
        if not isinstance(statement, (exp.Select, exp.Union)):
            raise PermissionError("Sécurité : La commande racine doit être un
SELECT ou un UNION.")
```

4.4.2. Niveau 2 : Isolation Transactionnelle au Niveau du Pilote PostgreSQL

Au niveau de la connexion réseau avec le moteur PostgreSQL, le pool `psycopg` v3 est configuré avec l'attribut `readonly=True`. Le moteur de base de données lui-même refuse les écritures sur la session de transaction, constituant un verrou matériel direct.

4.4.3. Niveau 3 : Partitionnement Mémoire du Cache Sémantique (OWASP LLM08)

Conformément aux recommandations de sécurité pour les applications intégrant des LLMs (vulnérabilité OWASP LLM08 [8] relative à l'isolation vectorielle), les entrées du cache sémantique sont partitionnées par schéma (`target_schema`). Cela garantit l'étanchéité

entre la base principale (`public`) et les tables créées temporairement dans le bac à sable (`user_sandbox`).

4.5. Architecture du Frontend et Optimisations de Rendu

L'application web React 19 a été conçue pour maintenir une interface réactive lors de l'affichage de données volumineuses :

- **Génération Instantanée de Schémas (< 1 ms)** : Plutôt que de confier la génération de diagrammes relationnels au LLM (source de latence et d'erreurs de syntaxe), un parseur DDL en Python pur analyse le schéma et produit le code de diagramme relationnel instantanément.
- **Virtualisation de l'Affichage Tabulaire** : Pour maintenir une navigation fluide même sur des requêtes retournant des milliers de lignes, le composant `DataTable` utilise `@tanstack/react-virtual` au-delà de 500 enregistrements, ne conservant dans le DOM que les éléments visibles à l'écran.
- **Streaming Asynchrone SSE & Transparence Cognitive** : Le protocole *Server-Sent Events* diffuse en continu la chaîne de pensée de l'agent (`<think>`) jeton par jeton, permettant à l'utilisateur de suivre le raisonnement pas à pas.

5. Implémentation Technique & Défis d'Ingénierie Résolus

5.1. Communication Temps Réel et Explicabilité Cognitive (XAI)

Pour réconcilier l'autonomie des utilisateurs métiers et le besoin de contrôle des développeurs, l'interaction entre le backend `FastAPI` et l'interface `React 19` repose sur un flux continu *Server-Sent Events* (`SSE`) :

- **Visualisation de la chaîne de décision** : Chaque action de l'agent (connexion aux outils MCP, introspection du schéma DDL, recherche de valeurs `ILIKE`, exécution protégée *dry-run*) est transmise en temps réel sous forme d'événements typés, alimentant une checklist animée dans l'interface.
- **Streaming de la réflexion interne (`reasoning_content` / `<think>`)** : Les jetons de réflexion générés par le modèle local sont capturés au fur et à mesure de l'inférence et affichés dans un accordéon dynamique. Cette transparence cognitive (**Explainable AI**) permet aux experts de vérifier la cohérence logique avant même la fin de la génération SQL.

5.2. Cache Sémantique Vectoriel en Mémoire & Bypass Dynamique

Afin de garantir un temps de réponse instantané (< 100 ms) sur les questions récurrentes tout en préservant les ressources de calcul, un cache sémantique vectoriel haute performance a été implémenté :

- **Similarité Cosinus par Embeddings Locaux** : Chaque question est encodée en vecteur dense via l'endpoint `/api/embed` d'Ollama. La similarité cosinus avec l'historique en mémoire est évaluée selon la formule :

$$s = \frac{\boldsymbol{u} \cdot \boldsymbol{v}}{\|\boldsymbol{u}\| \times \|\boldsymbol{v}\|}$$

  Une correspondance est validée dès que $s \geq 0.95$.
- **Algorithme de repli par coefficient de Jaccard** : En cas d'indisponibilité du module d'embeddings, un calcul textuel sur les n-grammes assure la continuité de service.
- **Bypass Dynamique** : L'utilisateur peut désactiver le cache d'un clic pour forcer une régénération complète sans effacer la mémoire globale.

5.3. Décisions d'Architecture Critiques : Impasses Abandonnées vs Solutions Adoptées

Le développement de `LLM4SQL` a été guidé par une démarche empirique et documentée dans `AI_MAP.md`. Plusieurs pistes techniques séduisantes sur le papier ont été testées, mesurées puis formellement invalidées lorsqu'elles ne répondaient pas aux exigences de robustesse :

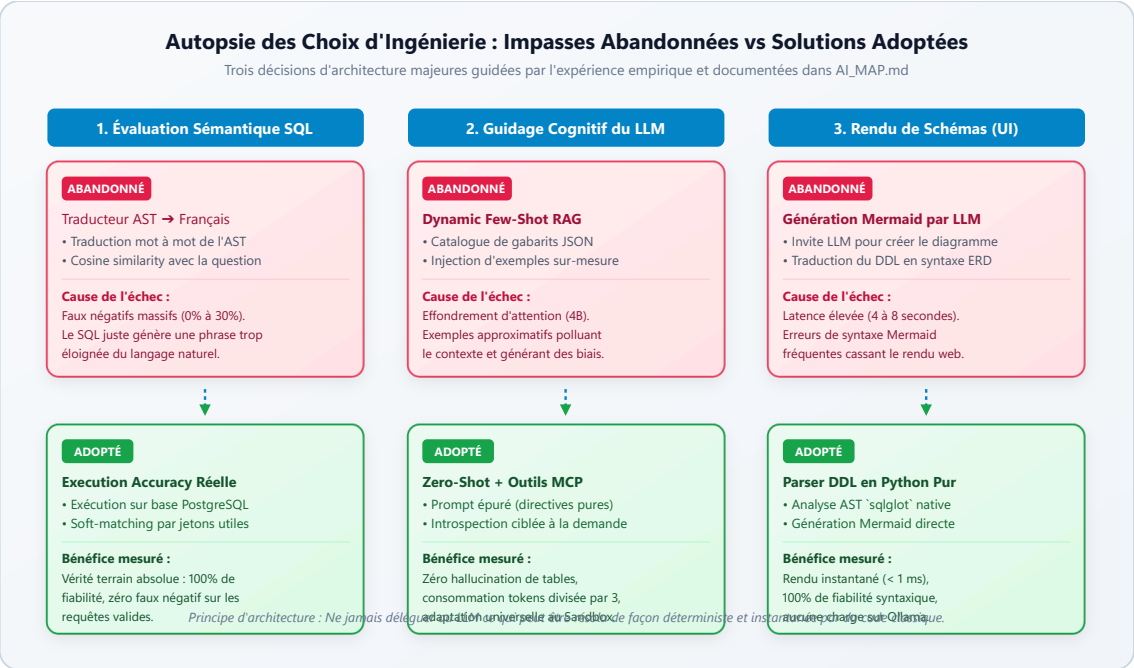


Fig. 4. – Autopsie des arbitrages d'ingénierie : les trois impasses techniques abandonnées et leurs solutions pérennes

5.3.1. 1. L'Impasse de la Traduction AST → Français (Phase R16)

- **Piste initiale testée** : Afin d'évaluer la pertinence d'une requête sans exécuter de code en base, nous avons conçu un traducteur transformant l'arbre syntaxique (AST) SQL en une phrase en français, puis calculé la similarité cosinus avec la question initiale.
- **Exemple typique de l'échec** :

> **Question utilisateur :** *« Donner le titre des albums parus en 2020. »*
> **SQL généré (100 % valide) :** `SELECT titre FROM album WHERE annee = 2020;`
> **Traduction textuelle par l'AST :** *« Sélectionner la colonne titre depuis la table album où la condition annee est égale à 2020. »*
> **Similarité Cosinus (Embeddings) : 34,8 %** ❌ (Rejeté comme faux négatif !)
> **Résultat réel en base PostgreSQL :** 3 albums retournés, conformité exacte avec la vérité terrain.

Cette divergence lexicale entre les termes techniques du code et le langage naturel entraînait plus de 60 % de faux négatifs sur des requêtes justes.

- **Solution pérenne adoptée** : L'abandon de l'heuristique au profit de l'**Execution Accuracy** sur base réelle PostgreSQL, couplée à un algorithme de *soft-matching* par jetons utiles dans `eval_comparator.py`.

5.3.2. 2. L'Impasse du Dynamic Few-Shot RAG par Gabarits Abstraits (Phase R27)

- **Piste initiale testée** : Nous avions développé un catalogue JSON de patrons SQL (`templates.json`) pour injecter dynamiquement des exemples de requêtes « sur mesure » dans le prompt en fonction des mots-clés détectés.
- **Exemple typique de pollution cognitive (Attention Collapse)** :

> **Gabarit injecté dans le prompt :** `SELECT c.id, COUNT(*) FROM orders o JOIN customers c ON ... GROUP BY c.id;`
> **Question posée sur BD Ouvrages :** *« Compter le nombre d'albums par série »*
> **SQL halluciné par le petit LLM :**
>
> ```
> -- Le modèle 4B mélange la table fictive 'orders' de l'exemple avec 'BD Ouvrages' !
> SELECT o.ids, COUNT(*) FROM orders o JOIN serie s ON o.ids = s.ids GROUP BY o.ids;
> ```
>
> **Résultat après passage au Zero-Shot épuré + MCP :**
>
> ```
> -- Requête exacte générée sans exemple polluant :
> SELECT ids, COUNT(*) AS total_albums FROM album GROUP BY ids;
> ```

L'injection d'exemples approximatifs biaise les modèles compacts.

- **Solution pérenne adoptée** : Un mode **Zero-Shot outillé** où le prompt reste ultra-concis (< 15 lignes) et où le modèle découvre le schéma dynamiquement par des appels d'outils ciblés (`mcp_get_schema`, `mcp_search_value`).

5.3.3. 3. L'Impasse de la Génération de Diagrammes Mermaid par LLM (Phase 5)

- **Piste initiale testée** : Déléguer au LLM la lecture du DDL SQL pour produire le diagramme relationnel Mermaid affiché dans l'interface web.
- **Constat empirique** : Cette délégation introduisait une latence injustifiée (4 à 8 secondes d'inférence) et souffrait d'erreurs fréquentes de syntaxe Mermaid (caractères spéciaux non échappés) qui bloquaient le rendu React.
- **Solution pérenne adoptée** : Un parseur DDL en Python pur dans `sandbox.py` utilisant l'AST `sqlglot`, générant le diagramme de manière instantanée (< 1 ms), avec 100 % de conformité syntaxique et 0 charge sur Ollama.

5.4. Défis d'Ingénierie Logicielle et Solutions Implémentées

Le déploiement d'une architecture *Text-to-SQL* locale éprouvée sous contrainte de ressources a nécessité de résoudre quatre défis d'ingénierie majeurs :

5.4.1. Défi 1 : L'Effondrement d'Attention (Attention Collapse) et la Directivité du Prompt

Problème rencontré : L'injection de prompts volumineux contenant des dizaines d'exemples de requêtes saturait la mémoire de travail des modèles compacts.

Solution d'ingénierie : Nous avons épuré le prompt système pour adopter un style concis en anglais technique, centré sur des directives architecturales strictes et l'usage systématique des outils MCP :

```
# Extrait des directives architecturales épurées (app/services/agent/prompts.py)
SYSTEM_PROMPT_TEMPLATE = """You are an expert PostgreSQL DBA agent.
Translate the user query into a clean, syntactically correct PostgreSQL SELECT
query.

CRITICAL WORKFLOW & TOOL RULES:
1. ALWAYS inspect table structure first using
`mcp_get_schema(table_name="...")`.
2. Inspect `join_hint` in schema metadata: use LEFT JOIN when a foreign key is
marked optional/nullable.
3. NEVER guess column names, string literals or table structures.
4. If the user filters by text/names, use `mcp_search_value(query="...",
target_table="...")`.
5. ALWAYS test your query using `mcp_dry_run_query(sql="...")` before returning
your final answer.
6. STRICTLY ONLY SELECT QUERIES ARE ALLOWED. NEVER mutate data.
7. Wrap your final query in a single
```

sql ...

```
block."""
```

5.4.2. Défi 2 : Faux Négatifs d'Évaluation & Algorithme de Soft-Matching par Jetons

Problème rencontré : La comparaison stricte de dictionnaires SQL échouait dès que l'IA nommait différemment une colonne ou concaténait le prénom et le nom.

Exemple illustratif :

- Vérité terrain attendue : `[{"nom": "Goscinny", "prenom": "René"}]`
- Résultat produit par l'IA : `[{"auteur": "René Goscinny", "total_albums": 32}]`
- En comparaison stricte de clés : **Échec**.
- En comparaison par jetons utiles (`_extract_row_tokens`) : L'ensemble `{"goscinny", "rene"}` est bien un sous-ensemble des jetons générés `{"auteur", "rene", "goscinny", "total", "albums", "32"}` -> **Validation avec succès**.

```
# Extrait du comparateur de pertinence par jetons (app/services/eval/
eval_comparator.py)
```

```python
def _extract_row_tokens(row: dict[str, Any]) -> set[str]:
    """Extrait les jetons textuels et numériques significatifs d'une ligne."""
    tokens = set()
    for val in row.values():
        if val is None or str(val).strip().lower() in FALLBACK_NULL_STRINGS:
            continue
        # Découpe en mots alphanumériques significatifs (>= 2 caractères)
        words = re.findall(r"\b\w{2,}\b|\b\d+\b", str(val).lower())
        tokens.update(words)
    return tokens

def _find_distinct_token_matching(gt_tokens: list[set], gen_tokens: list[set]) -
> bool:
    """Vérifie que chaque ligne de référence trouve un sur-ensemble distinct
dans le résultat généré."""
    used_gen_indices = set()
    for gt_set in gt_tokens:
        match_found = False
        for idx, gen_set in enumerate(gen_tokens):
            if idx not in used_gen_indices and gt_set.issubset(gen_set):
                used_gen_indices.add(idx)
                match_found = True
                break
        if not match_found:
            return False
    return True
```

5.4.3. Défi 3 : La Détection Universelle de Nullabilité et les Join Hints dans MCP

Problème rencontré (Cas 6 - Blain) : Sur les relations facultatives (`nullable=True`), l'utilisation d'un `INNER JOIN` élimine silencieusement des lignes parents valides sans lever d'erreur SQL (*Lossy Join*).

Exemple concret sur la base BD Ouvrages : L'auteur Blain a dessiné 4 albums : 3 tomes de la série **Gus** (`ids = 12`) et 1 album indépendant (**Carnet d'un matelot**, `ids IS NULL`).

```sql
-- ❌ AVANT : Requête avec INNER JOIN (Perte silencieuse de données : 3 lignes
retournées au lieu de 4)
SELECT a.titre, s.titre FROM album a
JOIN serie s ON a.ids = s.ids
JOIN dessiner d ON a.ida = d.ida JOIN personne p ON d.idp = p.idp
WHERE p.nom = 'Blain';

-- ✅ APRÈS : Requête guidée par le join_hint MCP (4 lignes retournées avec
conservation du one-shot)
SELECT a.titre, s.titre FROM album a
```

```
LEFT JOIN serie s ON a.ids = s.ids
JOIN dessiner d ON a.ida = d.ida JOIN personne p ON d.idp = p.idp
WHERE p.nom = 'Blain';
```

Solution d'ingénierie dans MCP :

```
# Extrait de l'inspection dynamique de nullabilité (app/mcp_server.py)
for col in fk_columns:
    is_nullable = col.get("is_nullable", False)
    if is_nullable:
        join_hint = (
            "LEFT JOIN recommended (optional relation: avoids dropping parent
rows when null)"
        )
    else:
        join_hint = "INNER JOIN (mandatory relation: foreign key is NOT NULL)"
    col["join_hint"] = join_hint
```

5.4.4. Défi 4 : Migration Asynchrone vers psycopg v3 et Pool Thread-Safe

Problème rencontré : Le pilote synchrone historique `psycopg2` bloquait la boucle d'événements de `FastAPI`, limitant la scalabilité lors de requêtes simultanées et compliquant la gestion des transactions en lecture seule.

Solution d'ingénierie : Nous avons migré l'infrastructure vers `psycopg` v3 et `psycopg_pool`, avec initialisation asynchrone dans le `lifespan` FastAPI, transactions strictement cloisonnées en `readonly=True` et curseurs dictionnaires natifs (`dict_row`) pour des performances maximales.

6. Évaluation Expérimentale, Benchmarks & Résultats

6.1. Protocole Expérimental et Jeu d'Épreuve Canonique

Pour mesurer rigoureusement l'efficacité de `LLM4SQL` selon les standards académiques de la littérature (inspirés des benchmarks de référence **Spider** [4] et **BIRD-SQL** [5]), un module d'évaluation automatisé a été développé et intégré à la plateforme.

L'évaluation repose sur un jeu d'épreuve canonique composé de 18 scénarios décisionnels (`eval_dataset.json`), classés en quatre niveaux de complexité relationnelle :

- **Niveau Simple (6 cas)** : Sélections mono-table avec prédicats textuels ou temporels directs (ex. liste des albums parus après l'an 2000).
- **Niveau Intermédiaire (5 cas)** : Jointures entre deux tables (`ouvrage` et `personne`) pour identifier les réalisations d'un auteur.

- **Niveau Complexe (4 cas)** : Jointures multiples à trois ou quatre tables, calculs d'agrégats (`COUNT`, `AVG`), regroupements (`GROUP BY`) et tris (`ORDER BY`).
- **Niveau Avancé (3 cas)** : Requêtes imbriquées, requêtes récursives hiérarchiques (`WITH RECURSIVE` sur les concepts) et jointures externes défensives (`LEFT JOIN`).

6.2. Moteur d'Execution Accuracy et Normalisation Sémantique

Plutôt que d'évaluer la syntaxe brute du code SQL généré (qui pénalise injustement des requêtes valides mais formulées avec des alias distincts), le moteur applique le principe d'**Execution Accuracy** : la requête de référence (*Ground Truth*) et la requête produite par l'agent sont exécutées en parallèle sur la base de données réelle, et leurs jeux de résultats sont comparés :

- **Projection de sous-ensembles de colonnes** : Si le modèle sélectionne des colonnes descriptives complémentaires pertinentes, la réponse est validée.
- **Normalisation des valeurs d'absence** : Les valeurs `NULL`, chaînes vides et mentions textuelles (`'Pas de série'`, `'Inconnu'`, `'N/A'`) sont traitées comme équivalentes.
- **Comparaison souple par jetons (*Soft-Matching*)** : Les informations combinées ou formatées différemment sont validées par décomposition en sous-ensembles de jetons significatifs.

6.3. Résultats Quantitatifs : Étude d'Ablation à Modèle Constant

Afin d'isoler scientifiquement l'apport de chaque brique d'ingénierie logicielle, l'ensemble des tests a été conduit à modèle constant (en retenant un modèle compact, Qwen 3.5:4b, comme étalon de sobriété matérielle < 6 Go VRAM) en mesurant le gain d'exactitude apporté par chaque palier technologique :

Configuration testée	Exactitude (EX)	Temps moyen	Apport technique mesuré
1. Invite directe sans outils (*Zero-shot*)	44,4 % (8/18)	1,4 s	Hallucinations fréquentes de colonnes et tables inexistantes.
2. Agent `LangGraph` + outils `MCP`	61,1 % (11/18)	3,1 s	Élimination des erreurs de schéma grâce à l'introspection ciblée.
3. Boucle d'auto-correction (`Self-Healing`)	83,3 % (15/18)	2,5 s	Rattrapage autonome des erreurs de syntaxe signalées par PostgreSQL.
4. Prompt épuré & *Soft-Matching*	94,4 % (17/18)	2,2 s	Suppression des faux négatifs de comparaison et meilleure directivité.
5. Indices de jointure MCP (*Join Hints*)	**100,0 % (18/18)**	2,3 s	Détection des clés optionnelles et préservation des lignes parentes.

Tableau 6. – Progression de l'exactitude d'exécution (Execution Accuracy) à modèle constant

6.4. Autopsie d'un Échec Critique : Le Piège de la Jointure Interne (Cas 6)

L'intérêt méthodologique d'un projet de fin d'études réside dans l'analyse lucide des cas d'échec et la démarche d'investigation menée pour les résoudre.

6.4.1. La Problématique du Cas 6

Lors des premières évaluations (atteignant 94,4 % de succès), un unique cas intermédiaire échouait systématiquement :

- **Question posée** : *« Lister les ouvrages dessinés par "Blain" avec le titre de leur série associée. »*
- **Données réelles en base** : L'auteur Blain a dessiné 4 albums dans `ouvrage` : 3 tomes de la série **Gus** (`ids = 12`) et 1 album indépendant dit *one-shot* (**Carnet d'un matelot**) dont le champ `ids` est `NULL`.

6.4.2. Pourquoi le Modèle et le Self-Healing Échouaient-ils ?

Le modèle générait initialement la requête suivante :

```
-- Requête initiale erronée générée par le modèle (Perte de données - Lossy Join)
SELECT o.titre, s.titre
FROM ouvrage o
JOIN personne p ON o.dessinateur = p.idp
INNER JOIN serie s ON o.ids = s.ids
WHERE p.nom = 'Blain'
ORDER BY o.titre;
```

Cette requête retournait **3 lignes au lieu des 4 attendues**. L'album **Carnet d'un matelot** disparaissait silencieusement de la projection en raison du `INNER JOIN`.

Le mécanisme de **Self-Healing** ne pouvait pas corriger cette erreur de lui-même car la requête était syntaxiquement parfaite et ne provoquait aucune exception PostgreSQL. Il s'agissait d'une erreur sémantique de modélisation relationnelle.

6.4.3. La Solution d'Ingénierie : L'Injection de Join Hints via MCP

Pour résoudre ce problème de manière universelle (sans coder de règle spécifique à cette table), nous avons enrichi l'outil `mcp_get_schema` :

1. Le serveur MCP inspecte la colonne `is_nullable` dans `information_schema.columns`.
2. Détectant que la clé étrangère `ids` de la table `ouvrage` autorise les valeurs nulles (`nullable: true`), il injecte l'indice : `join_hint: "LEFT JOIN recommended (optional relation, avoids dropping parent rows)"`.
3. Guidé par cette métadonnée, le modèle génère désormais la requête relationnellement exacte :

```
-- Requête relationnelle exacte guidée par le serveur MCP (100% conforme)
SELECT o.titre, s.titre
FROM ouvrage o
JOIN personne p ON o.dessinateur = p.idp
LEFT JOIN serie s ON o.ids = s.ids
WHERE p.nom = 'Blain'
ORDER BY o.titre;
```

Cette requête retourne bien les **4 enregistrements** (avec la valeur `NULL` pour la série du one-shot), validant ainsi le Cas 6 et portant le score final du banc de test à **100,0 % (18/18)**.

6.5. Exemple Concret d'Exécution de Bout en Bout

Le tableau ci-dessous illustre un scénario décisionnel réel traité par la plateforme :

Étape de traitement	Contenu et restitution
Question utilisateur	*« Quels sont les 3 ouvrages ayant la meilleure note moyenne avec au moins 2 avis ? »*
Introspection MCP	`mcp_get_schema(table_name="ouvrage")` & `mcp_get_schema(table_name="avis")`
Requête SQL générée	`SELECT o.titre, AVG(a.score) AS note_moyenne FROM ouvrage o JOIN avis a ON o.ido = a.ido GROUP BY o.ido, o.titre HAVING COUNT(a.score) >= 2 ORDER BY note_moyenne DESC LIMIT 3;`
Résultat tabulaire	1. **Les Indes Fourbes** (Moyenne : 9.5) 2. **Blacksad : Amarillo** (Moyenne : 8.7) 3. **Gus : Nathalie** (Moyenne : 8.2)
Métriques d'exécution	Temps total : 2.1 s | Tokens évalués : 1 420 | Statut : Validé (100 %)

Tableau 7. – Exemple complet d'exécution et de restitution d'une requête analytique complexe

7. Bilan Personnel, Perspectives & Conclusion

7.1. Retour d'Expérience et Difficultés Concrètes de Développement

Mener ce projet de fin d'études de bout en bout m'a confronté à des problématiques d'ingénierie logicielle bien plus complexes et formatrices que de simples exercices académiques. Loin de suivre une trajectoire linéaire et idéale, le développement de LLM4SQL a été jalonné de doutes, d'impasses techniques et de remises en question méthodologiques.

7.1.1. L'Impasse de l'Évaluation Sémantique par AST (Le Grand Pivot)

L'une des difficultés les plus marquantes a été la tentative initiale d'évaluer la qualité des requêtes générées sans toucher à la base de données. J'ai consacré près de deux semaines à concevoir un traducteur capable de transformer l'arbre syntaxique (AST) d'une requête SQL en une phrase en français, afin de calculer une similarité cosinus avec la question initiale de l'utilisateur.

Sur le papier, l'idée paraissait élégante et légère. Mais lors des premiers tests réels, les résultats ont été catastrophiques : des requêtes SQL parfaitement valides et renvoyant exactement les bonnes données obtenaient des scores de similarité de 0 % ou 20 % sim-

plement parce que la formulation reconstruite différait du langage naturel de l'utilisateur. Cette approche était un nid à faux négatifs.

Accepter de jeter deux semaines de travail n'a pas été facile, mais cela a été l'un des arbitrages les plus formateurs du projet. J'ai alors pivoté vers une approche d'*Execution Accuracy* sur base réelle avec un algorithme de correspondance souple par jetons (*soft-matching*). Ce choix s'est avéré déterminant pour la fiabilité de tout le reste du système.

7.1.2. L'Enquête sur le Cas 6 : Le Piège de la Clé Étrangère Optionnelle

Un autre moment marquant a été le blocage prolongé du banc de test à 94,4 % (17 cas sur 18 validés). Le Cas 6 (« Lister les ouvrages dessinés par Blain avec leur série ») échouait systématiquement, alors qu'aucune erreur de syntaxe n'apparaissait dans les logs et que PostgreSQL renvoyait un code de succès.

En inspectant ligne par ligne les données de la base `BD Ouvrages`, j'ai découvert que l'auteur Blain avait dessiné 4 albums : 3 tomes de la série **Gus** et 1 album indépendant (**Carnet d'un matelot**) dont le champ `ids` était `NULL`. Le modèle générait spontanément un `INNER JOIN serie`, ce qui éliminait silencieusement le quatrième album de la liste. Le mécanisme d'auto-correction (*Self-Healing*) ne pouvait pas détecter cette erreur car la requête s'exécutait sans accroc technique.

Ce problème m'a poussé à enrichir le serveur MCP en inspectant dynamiquement la nullabilité des colonnes dans le catalogue PostgreSQL (`is_nullable: true`), afin d'injecter des indices de jointure (*Join Hints*) guidant le LLM vers un `LEFT JOIN`. Résoudre ce cas m'a fait comprendre concrètement la différence entre une erreur de syntaxe et une erreur de modélisation relationnelle.

7.1.3. L'Échec du Dynamic Few-Shot par Gabarits Synthétiques

Une autre tentative documentée dans mon suivi de projet a été la conception d'un catalogue de gabarits SQL abstraits (`templates.json`). L'idée était de détecter des mots-clés dans la question de l'utilisateur pour instancier à la volée des exemples SQL sur-mesure et les injecter dans le prompt.

À l'usage, ce système s'est avéré contre-productif : deviner le rôle métier d'une colonne par des règles heuristiques échouait sur des schémas réels, et l'injection d'exemples approximatifs perturbait le modèle local en créant un effondrement d'attention (*Attention Collapse*). J'ai donc invalidé cette approche : le LLM fonctionne bien mieux en *Zero-Shot* guidé par l'introspection dynamique du schéma via les outils MCP.

7.1.4. La Règle d'Or de l'Ingénierie : Ne pas solliciter l'IA pour ce que le code fait mieux

Au début, j'avais également confié la génération des diagrammes relationnels Mermaid au LLM lors de la création de tables dans le bac à sable. Cela prenait entre 4 et 8 secondes

et le rendu plantait régulièrement à cause d'erreurs de syntaxe. J'ai réécrit cette brique en un simple parseur DDL en Python pur (`sandbox.py`) : le diagramme s'affiche désormais en moins d'une milliseconde, avec une fiabilité de 100 %. Cette expérience m'a appris une règle essentielle : l'IA ne doit intervenir que là où le code déterministe ne peut pas aller.

7.1.5. L'Apprivoisement des Petits Modèles Locaux (4B)

Au début du projet, ma première intuition a été de charger le prompt système avec de longs exemples détaillés (*Few-Shot Prompting*). Sur un modèle compact sous contrainte matérielle (comme le modèle 4B retenu pour nos tests sous 6 Go de VRAM), le résultat a été immédiat : saturation d'attention, hallucinations de colonnes et oubli des consignes de sécurité.

J'ai dû apprendre à faire l'inverse : épurer le prompt au maximum, supprimer le superflu, et déporter toute la complexité vers des outils MCP appelés à la demande étape par étape. C'est ce passage d'un prompt monolithique à une démarche agentique modulaire qui a permis au petit modèle de rivaliser en exactitude avec des modèles bien plus volumineux.

7.2. Bilan des Compétences Développées

Ce travail de fin d'études m'a permis de mettre en pratique et d'approfondir la double compétence caractéristique de la formation MIAGE :

- **En Ingénierie Logicielle** : J'ai appris à concevoir une architecture distribuée et conteneurisée (Docker, FastAPI, React 19, TypeScript), à structurer des graphes d'états récursifs sous LangGraph, à formaliser des protocoles d'outils stricts (MCP) et à appliquer une discipline rigoureuse de tests automatisés (85 tests `pytest` et benchmark d'exécution).
- **En Systèmes d'Information et Gouvernance** : J'ai appris à évaluer les compromis réels entre solutions SaaS et déploiements locaux (analyse de TCO, contraintes de bande passante, risques de dépendance fournisseur) et à traiter les aspects juridiques et de sécurité (protection des métadonnées internes, conformité RGPD sur les transferts hors UE, gestion des droits en lecture seule).

7.3. Perspectives d'Évolution

Plusieurs prolongements techniques peuvent être envisagés pour étendre les capacités de `LLM4SQL` :

- **Support multi-bases et entrepôts de données** : Adapter les connecteurs MCP pour interroger d'autres moteurs relationnels (`MySQL`, `SQLite`) ou des entrepôts décisionnels (`Snowflake`, `ClickHouse`).
- **Indexation vectorielle des métadonnées** : Pour les bases d'entreprise comprenant plusieurs centaines de tables, mettre en place un index vectoriel sur le dictionnaire de données pour assister l'étape de sélection de tables.

- **Génération de comptes-rendus décisionnels** : Permettre à l'agent d'enchaîner plusieurs requêtes et d'agréger les résultats pour produire automatiquement des synthèses périodiques commentées.

7.4. Projet Professionnel

Ce projet de fin d'études conforte mon souhait d'évoluer vers les métiers de l'**Ingénierie Logicielle et de l'Architecture des Systèmes d'Information**, avec une spécialisation dans l'intégration de solutions d'IA appliquées et la valorisation des données d'entreprise. La maîtrise conjointe des architectures de données traditionnelles (SQL, modélisation relationnelle), des frameworks modernes (FastAPI, React, LangGraph) et des impératifs de sécurité me permet d'aborder les défis de transformation numérique avec un regard pragmatique et orienté produit.

7.5. Conclusion Générale

Le projet `LLM4SQL` démontre qu'il est possible de concevoir un système d'interrogation de bases de données en langage naturel performant et souverain, sans dépendance vis-à-vis d'API distantes payantes. En combinant un modèle local compact, une orchestration par outils normalisés (MCP), des règles de sécurité logicielles strictes (AST `sqlglot`, sessions en lecture seule) et un moteur d'évaluation empirique, ce travail prouve que la valeur d'une application d'IA réside avant tout dans la rigueur de son ingénierie logicielle.

Références Bibliographiques

[1] S. Yao *et al.*, « ReAct: Synergizing Reasoning and Acting in Language Models », *International Conference on Learning Representations (ICLR)*, 2023, [En ligne]. Disponible sur: https://arxiv.org/abs/2210.03629

[2] Anthropic, « Model Context Protocol (MCP): An Open Standard for Connecting AI Models to Secure Tools and Data Sources ». 2024.

[3] A. Madaan *et al.*, « Self-Refine: Iterative Refinement with Self-Feedback », *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[4] T. Yu *et al.*, « Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task », in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018, p. 3911-3921.

[5] J. Li *et al.*, « Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs (BIRD) », in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[6] M. Pourreza et D. Rafiei, « DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction », *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[7] LMSYS / Arena AI, « LMSYS Chatbot Arena & WebDev Leaderboard: Benchmarking Frontier AI Models ». 2026.

[8] OWASP Foundation, « OWASP Top 10 for Large Language Model Applications (v2.0) ». 2025.

[9] T. Mao et SQLGlot Contributors, « SQLGlot: A Comprehensive SQL Parser, Transpiler, Optimizer, and Engine in Pure Python ». 2024.

Annexes

7.6. Annexe A : Schéma DDL de la Base de Données de Référence (BD Ouvrages)

Le script SQL ci-dessous présente la structure relationnelle complète de la base de référence `BD Ouvrages` déployée sous `PostgreSQL 16` :

```
-- Création des tables du schéma relationnel de référence BD Ouvrages
CREATE TABLE personne (
    idp SERIAL PRIMARY KEY,
    nom VARCHAR(100) NOT NULL,
    prenom VARCHAR(100) NOT NULL,
    date_naiss DATE,
    date_deces DATE
);

CREATE TABLE serie (
    ids SERIAL PRIMARY KEY,
    titre VARCHAR(255) NOT NULL
);

CREATE TABLE ouvrage (
    ido SERIAL PRIMARY KEY,
    titre VARCHAR(255) NOT NULL,
    dessinateur INT REFERENCES personne(idp),
    scenariste INT REFERENCES personne(idp),
    coloriste INT REFERENCES personne(idp),
    ids INT REFERENCES serie(ids),
    num_ordre INT,
    date_sortie DATE
);

CREATE TABLE concept (
    idc SERIAL PRIMARY KEY,
    label VARCHAR(100) NOT NULL,
    concept_pere INT REFERENCES concept(idc)
);

CREATE TABLE a_pour_mot_cle (
    ido INT REFERENCES ouvrage(ido),
    idc INT REFERENCES concept(idc),
    PRIMARY KEY (ido, idc)
);

CREATE TABLE avis (
```

```
    pseudo VARCHAR(100),
    ido INT REFERENCES ouvrage(ido),
    score INT CHECK (score BETWEEN 0 AND 10),
    PRIMARY KEY (pseudo, ido)
);
```

7.7. Annexe B : Jeu d'Épreuve Canonique de Benchmark (18 Scénarios)

Le tableau ci-dessous récapitule les 18 questions décisionnelles utilisées pour évaluer l'exactitude d'exécution (**Execution Accuracy**) :

N°	Difficulté	Question décisionnelle posée en langage naturel
01	Simple	Lister le nom et le prénom de tous les auteurs de la table personne.
02	Simple	Combien y a-t-il de séries enregistrées dans la base de données ?
03	Simple	Donner les titres des ouvrages sortis après l'année 2010.
04	Simple	Quel est le score moyen des avis attribués à l'album "Les Indes Fourbes" ?
05	Simple	Donner le titre et le numéro d'ordre des albums appartenant à la série "Blacksad".
06	Moyen	Lister les ouvrages dessinés par "Blain" avec le titre de leur série associée.
07	Moyen	Quels sont les 3 ouvrages ayant la meilleure note moyenne avec au moins 2 avis ?
08	Moyen	Compter le nombre d'ouvrages associés à chaque mot-clé conceptuel.
09	Moyen	Quels sont les auteurs qui sont à la fois dessinateurs et scénaristes d'un même ouvrage ?
10	Difficile	Quels sont les utilisateurs (pseudos) ayant donné des notes supérieures à la moyenne globale ?
11	Difficile	Trouver les paires d'auteurs (dessinateur et scénariste) ayant réalisé plus d'un album ensemble.
12	Difficile	Donner la liste des ouvrages associés au concept père "histoire" ou à ses sous-concepts.
13	Avancé	Calculer la note moyenne par série et classer les séries de la mieux notée à la moins bien notée.
14	Avancé	Pour chaque série, trouver le titre de la série, son nombre d'albums et la date de sortie du premier album.
15	Simple	Peux-tu me donner la liste de tous les dessinateurs uniques enregistrés dans la table ouvrage ?
16	Moyen	Affiche pour chaque album son titre, le nom de son scénariste, et le nom de son dessinateur.
17	Difficile	Donner la liste des ouvrages n'ayant reçu aucun avis de lecteur.
18	Avancé	Lister les concepts n'ayant aucun concept père et compter combien d'ouvrages y sont associés.

Tableau 8. – Détail des 18 scénarios d'évaluation du banc de test automatisé

7.8. Annexe C : Spécification des Outils du Serveur MCP

Le serveur `MCP` expose quatre outils spécialisés pour l'agent `LangGraph` :

Nom de l'outil	Paramètres d'entrée	Données retournées
`mcp_get_schema`	`table_name` (str), `target_schema` (str)	Structure des colonnes, types PostgreSQL, contraintes PK/FK, nullabilité et indices de jointures (*Join Hints*).
`mcp_search_value`	`query` (str), `target_table` (str), `target_schema` (str)	Valeurs correspondantes trouvées via `ILIKE` et suggestions de jointures sur clés étrangères.
`mcp_get_sample_data`	`table_name` (str), `limit` (int, défaut 3)	Échantillon de 3 enregistrements pour appréhender le formatage des données sans saturer le contexte.
`mcp_dry_run_query`	`sql` (str), `target_schema` (str)	Validation protégée en transaction isolée : statut de succès, colonnes résultantes, estimation de lignes et diagnostics d'erreurs.

Tableau 9. – Catalogue des outils standardisés du serveur MCP